

Guía de estudio sobre los códigos en GNU Octave Usados para el ejemplo ilustrativo del Filtrado de Amplitudes mediante DFT

Kelvin A. P. H.

1. Introducción

Este documento tiene un propósito personal de estudio. Su objetivo es explicar con detalle la función de cada bloque de código utilizado en GNU Octave para la construcción de las figuras de la tesis. La idea no es solo saber qué hace cada línea, sino comprender por qué se usa y cómo se conecta con la teoría de Fourier.

La parte central del método es la FFT, que es un algoritmo eficiente para calcular la DFT. En lugar de calcular directamente todas las sumas de la transformada discreta, la FFT permite obtener los coeficientes espectrales de manera mucho más rápida. Por eso, en los códigos se usa la instrucción `fft(f_j)` como herramienta principal de análisis espectral. Luego, `fftshift` e `ifftshift` se emplean para reorganizar los coeficientes, y `ifft` permite reconstruir la señal en el dominio físico. [1]

2. Bloque común de espectro

```
1 % Calculo da Transformada de Fourier Discreta por meio da
  FFT
2 A = fft(f_j) / N;
3 % Reorganizacao dos coeficientes para centralizar n = 0
4 A_shift = fftshift(A);
5 % Indices frequenciais simetricos.
6 n_vals = -m:m;
7 % Amplitudes espectrais
8 amplitudes = abs(A_shift);
```

La línea `A = fft(f_j) / N;` calcula la Transformada de Fourier Discreta de las muestras `f_j`. La FFT no cambia la teoría, solo la forma de cálculo: sigue siendo la DFT, pero implementada por un algoritmo rápido. El factor `/ N` se usa porque en el trabajo se adopta una normalización de los coeficientes de Fourier discretos. [1]

La instrucción `A_shift = fftshift(A)`; reordena el espectro para que la frecuencia cero quede en el centro. Esto permite ver los modos negativos a la izquierda y los positivos a la derecha. Es una operación importante porque hace más natural la lectura simétrica del espectro. [1]

La línea `n_vals = -m:m`; construye el eje de frecuencias discretas. Si N es impar, entonces $N = 2m+1$, y el espectro puede representarse como $-m, \dots, 0, \dots, m$. Finalmente, `amplitudes = abs(A_shift)`; toma el módulo de los coeficientes, porque la figura representa la intensidad de cada frecuencia y no su fase.

3. Sinal amostrado

```

1 clear all
2 close all
3 graphics_toolkit('gnuplot');
4 N = 63;
5 j = 0:N-1;
6 x_j = (2*pi/N) * j;
7 f_j = sin(x_j) + (1/2)*sin(3*x_j) + (1/3)*sin(7*x_j) ...
8       + 0.3*sin(15*x_j) + 0.2*sin(20*x_j);
9
10 cor = [0.0000 0.4470 0.7410];
11 figure;
12 set(gcf, 'Color', 'w');
13 hold on
14
15 plot(x_j, f_j, '-.', 'LineWidth', 1.6, 'Color', cor);
16 plot([0, 2*pi], [0, 0], 'k-', 'LineWidth', 0.5);
17
18 grid on
19 box on
20 xlim([0, 2*pi]);
21 ylim([-1.6, 1.6]);
22 set(gca, 'YTick', [-1, 0, 1]);
23 set(gca, 'XTick', [0, pi/2, pi, 3*pi/2, 2*pi]);
24 set(gca, 'XTickLabel', {'0', '\pi/2', '\pi', '3\pi/2', '2\pi'});
25
26 xlabel('x');
27 ylabel('f(x_j)');
28
29 legend({'f(x_j) (N = 63 amostras, \Delta x \approx 0.10 rad)'}, ...
30        'Location', 'northoutside', ...
31        'Orientation', 'horizontal');
32 legend boxoff
33
34 hold off
35

```

```

36 set(gcf, 'PaperUnits', 'centimeters');
37 set(gcf, 'PaperPosition', [0 0 16 10]);
38 set(gcf, 'PaperSize', [16 10]);

```

Los comandos `clear all` y `close all` se usan para empezar desde un estado limpio: se eliminan variables previas y se cierran las figuras abiertas. Esto evita que resultados anteriores interfieran con la ejecución actual.

La instrucción `graphics_toolkit('gnuplot')`; selecciona el motor gráfico que se usará para generar la figura. En algunos entornos de Octave, esto ayuda a obtener resultados más estables al exportar.

La línea `N = 63;` define el número de muestras. Se toma un valor impar para que el espectro pueda organizarse simétricamente alrededor de la frecuencia cero. Luego, `j = 0:N-1;` genera los índices discretos y `x_j = (2*pi/N) * j;` construye la malla uniforme de puntos en $[0, 2\pi)$.

La expresión que define `f_j` genera la señal de prueba. Las frecuencias bajas representan la parte limpia y las altas representan el ruido. Esto permite observar claramente el efecto del filtrado posterior.

La variable `cor` define el color azul de la curva. No cambia el contenido matemático, pero ayuda a mantener consistencia visual entre las figuras. La instrucción `figure;` abre una nueva ventana gráfica y `set(gcf, 'Color', 'w');` fija el fondo blanco, lo cual es más limpio para una tesis.

`hold on` permite superponer varias curvas en la misma figura. Luego, `plot(x_j, f_j, ...)` dibuja la señal muestreada como una línea continua. La línea `plot([0, 2*pi], [0, 0], ...)` agrega el eje horizontal de referencia en $y = 0$.

Los comandos `grid on` y `box on` activan la cuadrícula y el marco del eje. `xlim` e `ylim` fijan los intervalos visibles. `set(gca, 'YTick', ...)` define las marcas del eje vertical, y `set(gca, 'XTick', ...)` coloca los puntos más importantes en el eje horizontal. `set(gca, 'XTickLabel', ...)` cambia esas marcas para que aparezcan como múltiplos de π , lo cual es más matemáticamente significativo. `set(gca, 'FontSize', 9);` reduce el tamaño de la fuente para que la figura sea más elegante y compacta.

Los comandos `xlabel` y `ylabel` colocan las etiquetas de los ejes. La `legend` identifica la curva y especifica el número de muestras y el paso Δx . Finalmente, `set(gcf, 'PaperUnits', ...)` y los dos comandos siguientes fijan el tamaño físico de exportación de la figura, para que el PDF salga con dimensiones controladas y compatibles con la tesis. [2]

4. Espectro de amplitudes

```

1 clear all
2 close all
3 graphics_toolkit('gnuplot');
4 N = 63;
5 m = 31;
6 j = 0:N-1;
7 x_j = (2*pi / N) * j;

```

```

8 f_j = sin(x_j) + (1/2)*sin(3*x_j) + (1/3)*sin(7*x_j) ...
9       + 0.3*sin(15*x_j) + 0.2*sin(20*x_j);
10
11 A = fft(f_j) / N;
12 A_shift = fftshift(A);
13 n_vals = -m:m;
14 amplitudes = abs(A_shift);
15
16 cor = [0.0000 0.4470 0.7410];
17 figure;
18 set(gcf, 'Color', 'w');
19 hold on
20
21 stem(n_vals, amplitudes, ...
22      'filled', ...
23      'Color', cor, ...
24      'MarkerSize', 4, ...
25      'LineWidth', 1.2);
26
27 grid on
28 box on
29 xlim([-m-1, m+1]);
30 ylim([0, 0.58]);
31
32 set(gca, 'XTick', [-20, -15, -10, -7, -3, -1, 0, 1, 3, 7,
33                  10, 15, 20]);
34 set(gca, 'YTick', [0, 0.1, 0.2, 0.3, 0.4, 0.5]);
35 set(gca, 'FontSize', 9);
36
37 xlabel('n');
38 ylabel('Amplitude');
39 hold off
40
41 set(gcf, 'PaperUnits', 'centimeters');
42 set(gcf, 'PaperPosition', [0 0 16 10]);
43 set(gcf, 'PaperSize', [16 10]);

```

Este código repite la generación de la señal porque necesita calcular su espectro. La línea `A = fft(f_j) / N;` convierte las muestras a coeficientes de Fourier discretos. La instrucción `A_shift = fftshift(A);` reorganiza esos coeficientes para que la frecuencia cero quede en el centro. `n_vals = -m:m;` define los índices espectrales simétricos y `amplitudes = abs(A_shift);` calcula la magnitud de cada modo.

La función `stem` se usa porque el espectro es discreto. Cada frecuencia se representa con una barra vertical, lo que permite visualizar claramente los picos. El color azul y el tamaño de los marcadores ayudan a mantener una apariencia homogénea con el resto de las figuras.

Los comandos `xlim`, `ylim`, `set(gca, ...)` y `xlabel/ylabel` cumplen aquí

el mismo papel que antes, pero adaptados al dominio de las frecuencias. El eje horizontal ahora se interpreta como índice frecuencial n , y el eje vertical como amplitud.

5. Espectro filtrado

```

1 clear all
2 close all
3 graphics_toolkit('gnuplot');
4 N = 63;
5 m = 31;
6 M = 10;
7 j = 0:N-1;
8 x_j = (2*pi / N) * j;
9 f_j = sin(x_j) + (1/2)*sin(3*x_j) + (1/3)*sin(7*x_j) ...
10     + 0.3*sin(15*x_j) + 0.2*sin(20*x_j);
11
12 A = fft(f_j) / N;
13 A_shift = fftshift(A);
14 n_vals = -m:m;
15
16 A_filtrado = zeros(1, N);
17 idx_passa = (m+1-M):(m+1+M);
18 A_filtrado(idx_passa) = A_shift(idx_passa);
19 amplitudes_filtrado = abs(A_filtrado);
20
21 cor = [0.4660 0.6740 0.1880];
22 figure;
23 set(gcf, 'Color', 'w');
24 hold on
25
26 stem(n_vals, amplitudes_filtrado, ...
27     'filled', ...
28     'Color', cor, ...
29     'MarkerSize', 4, ...
30     'LineWidth', 1.2);
31
32 xline(M, '--r', 'LineWidth', 1.0);
33 xline(-M, '--r', 'LineWidth', 1.0);
34
35 grid on
36 box on
37 xlim([-m-1, m+1]);
38 ylim([0, 0.58]);
39
40 set(gca, 'XTick', [-20, -15, -10, -7, -3, -1, 0, 1, 3, 7,
41     10, 15, 20]);
42 set(gca, 'FontSize', 9);

```

```

42 set(gca, 'YTick', [0, 0.1, 0.2, 0.3, 0.4, 0.5]);
43
44 xlabel('n');
45 ylabel('Amplitude');
46 text(10.5, 0.54, 'M=10', 'FontSize', 8, 'Color', 'r');
47
48 hold off
49
50 set(gcf, 'PaperUnits', 'centimeters');
51 set(gcf, 'PaperPosition', [0 0 16 10]);
52 set(gcf, 'PaperSize', [16 10]);

```

Aquí aparece el filtro espectral. Primero se inicializa `A_filtrado = zeros(1, N)`; para crear un vector lleno de ceros. Esto significa que, por defecto, se van a eliminar todas las frecuencias. Luego `idx_passa = (m+1-M):(m+1+M)`; selecciona el bloque central de coeficientes que corresponde a $|n| \leq M$. Como el índice central es $m + 1$, se toma un intervalo simétrico alrededor de ese punto.

La línea `A_filtrado(idx_passa) = A_shift(idx_passa)`; copia únicamente las componentes de baja frecuencia y deja en cero las demás. Así se implementa el filtro pasa-bajas ideal. Las líneas `xline(M,...)` y `xline(-M,...)` marcan visualmente el umbral de corte. El texto `M=10` refuerza la lectura de la figura.

6. Reconstrução filtrada

```

1  clear all
2  close all
3  graphics_toolkit('gnuplot');
4  N = 63;
5  m = 31;
6  M = 10;
7  j = 0:N-1;
8  x_j = (2*pi / N) * j;
9  f_j = sin(x_j) + (1/2)*sin(3*x_j) + (1/3)*sin(7*x_j) ...
10      + 0.3*sin(15*x_j) + 0.2*sin(20*x_j);
11
12  A = fft(f_j) / N;
13  A_shift = fftshift(A);
14  n_vals = -m:m;
15
16  A_filtrado = zeros(1, N);
17  idx_passa = (m+1-M):(m+1+M);
18  A_filtrado(idx_passa) = A_shift(idx_passa);
19
20  A_filtrado_orig = ifftshift(A_filtrado);
21  a_reconstruido = real(ifft(A_filtrado_orig) * N);
22
23  x_cont = linspace(0, 2*pi, 1000);

```

```

24 f0_cont = sin(x_cont) + (1/2)*sin(3*x_cont) + (1/3)*sin(7*
    x_cont);
25
26 cor_ruidoso = [0.6000 0.7800 0.9200];
27 cor_limpo = [0.8500 0.3250 0.0980];
28 cor_reconstr = [0.4660 0.6740 0.1880];
29 figure;
30 set(gcf, 'Color', 'w');
31 hold on
32
33 plot(x_j, f_j, '-', 'LineWidth', 1.0, 'Color', cor_ruidoso);
34 plot(x_cont, f0_cont, '-.-', 'LineWidth', 1.8, 'Color',
    cor_limpo);
35 plot(x_j, a_reconstruido, '-', 'LineWidth', 1.8, 'Color',
    cor_reconstr);
36 plot([0, 2*pi], [0, 0], 'k-', 'LineWidth', 0.5);
37
38 grid on
39 box on
40 xlim([0, 2*pi]);
41 ylim([-1.6, 1.6]);
42
43 set(gca, 'XTick', [0, pi/2, pi, 3*pi/2, 2*pi]);
44 set(gca, 'XTickLabel', {'0', '\pi/2', '\pi', '3\pi/2', '2\pi
    '});
45 set(gca, 'YTick', [-1, 0, 1]);
46 set(gca, 'FontSize', 9);
47
48 xlabel('x');
49 ylabel('');
50
51 legend({'f(x_j) (ruidoso)', ...
52         'f_0(x) (limpo)', ...
53         'Reconstrucao filtrada'}, ...
54         'Location', 'northoutside', ...
55         'Orientation', 'horizontal');
56 legend boxoff
57
58 hold off
59
60 set(gcf, 'PaperUnits', 'centimeters');
61 set(gcf, 'PaperPosition', [0 0 16 10]);
62 set(gcf, 'PaperSize', [16 10]);

```

Este código cierra el ciclo: primero se filtra el espectro y luego se reconstruye la señal filtrada. La instrucción `A_filtrado_orig = ifftshift(A_filtrado)`; deshace la reorganización aplicada por `fftshift`. Después `ifft` calcula la transformada inversa y recupera la señal en el dominio físico. El factor `* N` compensa la normalización usada al principio.

La función `real(...)` elimina una posible parte imaginaria residual causada por errores numéricos de redondeo. La curva `f0_cont` se construye para comparar la reconstrucción con la señal limpia ideal. Los tres colores distinguen la señal ruidosa, la señal limpia y la señal reconstruida. La línea horizontal en $y = 0$ ayuda a ver mejor la forma general de las curvas.

7. Resumen para recordar

Para estudiar la defensa, conviene memorizar esta secuencia:

1. Se construyen las muestras `f_j`.
2. Se calcula el espectro con `fft`.
3. Se reordena con `fftshift`.
4. Se filtra eliminando altas frecuencias.
5. Se reconstruye con `ifftshift` e `ifft`.

Las instrucciones de formato gráfico como `set(gca, ...)` y `set(gcf, ...)` no cambian la matemática, pero son esenciales para que las figuras se presenten correctamente en el PDF final. En particular, `PaperUnits`, `PaperPosition` y `PaperSize` fijan el tamaño físico de la figura al exportarla, evitando problemas de escala o márgenes excesivos. [2]

Referencias

- [1] GNU Octave Documentation, *fftshift* and *ifftshift* / plotting and saving plots.
- [2] GNU Octave Documentation, *Printing and Saving Plots*.